

UNIVERSIDAD NACIONAL DE SAN AGUSTÍN

Escuela Profesional de Ciencia de la Computación

COMPUTACIÓN GRÁFICA

Laboratorio 8

Iluminación y Texturizado

en OpenGL

Estudiante: Rivas Huanca Diego Raul

`drivash@unsa.edu.pe`

8 de junio de 2026

Índice

1. Fundamentos de Iluminación	2
1.1. Conceptos generales	2
1.1.1. ¿Qué es la iluminación en computación gráfica?	2
1.1.2. ¿Por qué un objeto sin iluminación se ve plano?	2
1.1.3. Diferencia entre color e iluminación	2
1.2. Componentes del modelo de iluminación clásico	2
1.2.1. Luz ambiente (Ambient)	2
1.2.2. Luz difusa (Diffuse)	3
1.2.3. Luz especular (Specular)	3
1.3. Normales en una superficie	3
1.3.1. Vector normal	3
1.3.2. Normales por vértice	3
1.3.3. Normales por cara	4
1.3.4. Efecto de normales mal definidas	4
1.4. Tipos de luces en OpenGL	4
2. Fundamentos de Texturizado	4
2.1. Conceptos básicos	4
2.1.1. ¿Qué es una textura?	4
2.1.2. ¿Qué es el mapeo de texturas?	5
2.1.3. ¿Qué son las coordenadas UV?	5
2.2. Proceso completo de texturizado	5
2.3. Formatos de imagen para texturas	6

1. Fundamentos de Iluminación

1.1. Conceptos generales

1.1.1. ¿Qué es la iluminación en computación gráfica?

La iluminación en computación gráfica es el proceso mediante el cual se simula el comportamiento de la luz sobre los objetos de una escena virtual. Su objetivo es reproducir fenómenos ópticos del mundo real como sombras, reflejos y degradados de color para generar imágenes con apariencia realista [1].

En OpenGL, la iluminación se calcula en cada vértice mediante el modelo de iluminación de Phong, que combina tres componentes: luz ambiente, luz difusa y luz especular.

1.1.2. ¿Por qué un objeto sin iluminación se ve plano?

Sin iluminación, todos los fragmentos de un objeto reciben el mismo color base, independientemente de su orientación respecto a la fuente de luz. Esto elimina la percepción de profundidad y curvatura que el ojo humano extrae de las variaciones de brillo sobre una superficie [2].

1.1.3. Diferencia entre color e iluminación

El **color** es una propiedad intrínseca del material que define qué longitudes de onda refleja. La **iluminación** determina cuánta luz de una fuente externa incide sobre la superficie y en qué dirección. El color final visible es el producto de ambos: el color del material modulado por la intensidad de luz que lo alcanza [3].

1.2. Componentes del modelo de iluminación clásico

El modelo de iluminación de Phong, implementado en OpenGL, descompone la luz en tres términos [4]:

$$I = I_a k_a + I_d k_d (\hat{n} \cdot \hat{l}) + I_s k_s (\hat{r} \cdot \hat{v})^n$$

donde I_a , I_d , I_s son las intensidades ambiente, difusa y especular; k_a , k_d , k_s los coeficientes del material; $\hat{n}$ la normal, $\hat{l}$ el vector a la luz, $\hat{r}$ el reflejo y $\hat{v}$ el vector al observador.

1.2.1. Luz ambiente (Ambient)

Definición: Simula la luz que ha rebotado tantas veces en el entorno que llega con intensidad uniforme desde todas las direcciones.

Función: Evita que las zonas sin iluminación directa queden completamente negras, añadiendo una iluminación mínima global.

En OpenGL:

```
1 float ambient[] = { 0.2f, 0.2f, 0.2f, 1.0f };  
2 glLightfv(GL_LIGHT0, GL_AMBIENT, ambient);
```

1.2.2. Luz difusa (Diffuse)

Definición: Luz que incide directamente sobre una superficie y se dispersa en todas las direcciones de forma uniforme (reflexión lambertiana).

Función: Produce la variación de brillo que otorga sensación de volumen y forma. Su intensidad depende del ángulo entre la normal de la superficie y el vector hacia la fuente de luz ($\hat{n} \cdot \hat{l}$).

En OpenGL:

```
1 float diffuse[] = { 0.8f, 0.8f, 0.8f, 1.0f };
2 glLightfv(GL_LIGHT0, GL_DIFFUSE, diffuse);
```

1.2.3. Luz especular (Specular)

Definición: Reflejo directo de la fuente de luz sobre superficies brillantes, produciendo un destello concentrado.

Función: Simula la apariencia metálica o plástica de los materiales. Su intensidad depende del ángulo entre el vector de reflexión y el vector al observador, elevado a una potencia n que controla el tamaño del destello.

En OpenGL:

```
1 float specular[] = { 1.0f, 1.0f, 1.0f, 1.0f };
2 glLightfv(GL_LIGHT0, GL_SPECULAR, specular);
3 glMaterialf(GL_FRONT, GL_SHININESS, 64.0f);
```

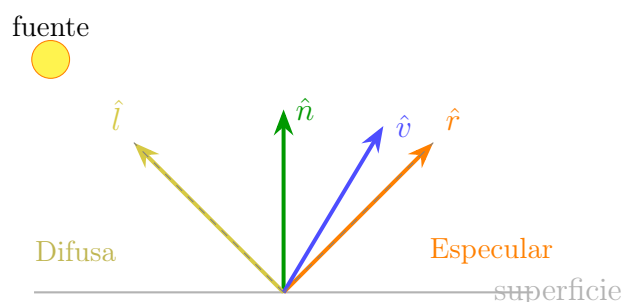


Figura 1: Vectores del modelo de iluminación de Phong: normal $\hat{n}$, vector a la luz $\hat{l}$, reflexión $\hat{r}$ y observador $\hat{v}$.

1.3. Normales en una superficie

1.3.1. Vector normal

Un vector normal es un vector perpendicular a una superficie en un punto dado. En computación gráfica, las normales son fundamentales para el cálculo de iluminación ya que determinan cómo la luz interactúa con cada punto de la superficie [1].

1.3.2. Normales por vértice

Las normales por vértice se definen en cada vértice de la malla. OpenGL interpola estos vectores entre vértices durante la rasterización, produciendo una apariencia suave (sombreado de Gouraud). Son adecuadas para superficies curvas como esferas.

```

1 glNormal3f(nx, ny, nz);    // normal antes del vertice
2 glVertex3f(vx, vy, vz);

```

1.3.3. Normales por cara

Las normales por cara asignan un único vector normal a toda la cara poligonal. Todos los fragmentos de esa cara reciben la misma iluminación, produciendo un sombreado facetado. Son apropiadas para objetos con aristas bien definidas como cubos.

1.3.4. Efecto de normales mal definidas

Cuando las normales están incorrectamente orientadas (por ejemplo, apuntando hacia el interior del objeto), la iluminación se calcula de forma errónea y las superficies aparecen completamente oscuras o con brillo invertido. Si las normales no se renormalizan tras aplicar transformaciones de escala, su longitud cambia y el cálculo de iluminación se distorsiona, produciendo zonas sobreiluminadas. Esto se corrige con `glEnable(GL_NORMALIZE)`.

1.4. Tipos de luces en OpenGL

OpenGL soporta hasta ocho fuentes de luz simultáneas (`GL_LIGHT0` a `GL_LIGHT7`). Los tipos principales son [3]:

Tipo	Posición	Característica
Direccional	$w = 0$	Rayos paralelos, simula el Sol
Puntual	$w = 1$	Emite en todas direcciones, tiene posición
Spot	$w = 1 + \text{dirección}$	Cono de luz con ángulo de corte

Cuadro 1: Tipos de fuentes de luz disponibles en OpenGL.

La componente w del vector de posición determina el tipo: $w = 0$ produce luz direccional y $w = 1$ produce luz puntual.

```

1 float posDir[]    = { 1.0f, 1.0f, 0.0f, 0.0f }; // direccional
2 float posPuntual[] = { 2.0f, 3.0f, 2.0f, 1.0f }; // puntual
3 glLightfv(GL_LIGHT0, GL_POSITION, posDir);

```

2. Fundamentos de Texturizado

2.1. Conceptos básicos

2.1.1. ¿Qué es una textura?

Una textura es una imagen bidimensional (o de mayor dimensión) que se aplica sobre la superficie de un objeto 3D para representar detalles visuales como colores, patrones, rugosidades o acabados superficiales sin incrementar la complejidad geométrica del modelo [1].

2.1.2. ¿Qué es el mapeo de texturas?

El mapeo de texturas es el proceso de asociar cada punto de una superficie 3D con un píxel (texel) de una imagen 2D. Este proceso transforma las coordenadas tridimensionales del objeto en coordenadas bidimensionales de la textura [2].

2.1.3. ¿Qué son las coordenadas UV?

Las coordenadas UV son un sistema de referencia bidimensional normalizado en el rango $[0, 1] \times [0, 1]$ utilizado para especificar qué parte de la textura corresponde a cada vértice de la geometría. La letra U corresponde al eje horizontal y V al eje vertical de la imagen [3].

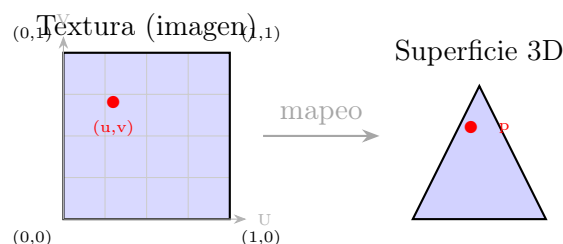


Figura 2: Mapeo de coordenadas UV: cada punto p de la superficie 3D se asocia con una coordenada (u, v) en la imagen de textura.

2.2. Proceso completo de texturizado

El proceso de texturizado en OpenGL sigue los siguientes pasos [1]:

1. **Imagen:** Se carga una imagen en memoria (PNG, JPG, BMP) y se decodifica en un arreglo de píxeles RGBA.
2. **Objeto de textura:** Se genera un identificador y se sube la imagen a la GPU:

```

1 GLuint texID;
2 glGenTextures(1, &texID);
3 glBindTexture(GL_TEXTURE_2D, texID);
4 glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, width, height,
5             0, GL_RGB, GL_UNSIGNED_BYTE, data);

```

3. **Parámetros de filtrado:** Se define cómo escalar la textura cuando es más pequeña o más grande que la superficie:

```

1 glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER,
2                 GL_LINEAR);
3 glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER,
4                 GL_LINEAR);

```

4. **Coordenadas UV:** Se asigna a cada vértice su coordenada de textura:

```

1 glEnable(GL_TEXTURE_2D);
2 glBindTexture(GL_TEXTURE_2D, texID);
3 glBegin(GL_QUADS);
4     glTexCoord2f(0,0); glVertex3f(-1,-1,0);
5     glTexCoord2f(1,0); glVertex3f( 1,-1,0);
6     glTexCoord2f(1,1); glVertex3f( 1, 1,0);
7     glTexCoord2f(0,1); glVertex3f(-1, 1,0);
8 glEnd();

```

5. **Superficie 3D:** La GPU interpola las coordenadas UV entre vértices y muestra el texel correspondiente en cada fragmento.

2.3. Formatos de imagen para texturas

Formato	Compresión	Transparencia	Uso recomendado
PNG	Sin pérdida	Sí (canal alfa)	Sprites, interfaces, texturas con transparencia
JPG	Con pérdida	No	Fotografías, fondos, texturas fotorrealistas
BMP	Sin compresión	Opcional	Compatibilidad máxima, archivos de gran tamaño

Cuadro 2: Comparación de formatos de imagen para texturas en OpenGL.

PNG es preferible cuando se requiere transparencia o calidad sin pérdida. JPG ofrece mayor compresión a costa de artefactos visibles en zonas de alto contraste. BMP no comprime, por lo que produce archivos grandes pero es universalmente compatible con OpenGL sin necesidad de decodificación especial [2].

Conclusiones

La iluminación en OpenGL se basa en el modelo de Phong que descompone la luz en tres componentes: ambiente, difusa y especular, cada una con un rol diferente en la apariencia final del objeto. Las normales son vectores fundamentales para este cálculo y deben estar correctamente orientadas y normalizadas.

El texturizado permite aumentar el realismo visual sin incrementar la complejidad geométrica, asociando cada vértice con coordenadas UV que mapean puntos de una imagen sobre la superficie. La elección del formato de imagen afecta la calidad, el tamaño en disco y la compatibilidad del recurso.

Referencias

- [1] Shreiner, D., Sellers, G., Kessenich, J., Licea-Kane, B. *OpenGL Programming Guide: The Official Guide to Learning OpenGL*. 9th ed. Addison-Wesley, 2016.
- [2] Shirley, P., Marschner, S. *Fundamentals of Computer Graphics*. 4th ed. CRC Press, 2015.

- [3] Angel, E., Shreiner, D. *Interactive Computer Graphics: A Top-Down Approach with WebGL*. 7th ed. Pearson, 2014.
- [4] Phong, B. T. “Illumination for Computer Generated Pictures.” *Communications of the ACM*, vol. 18, no. 6, pp. 311–317, 1975.